k8s1.30部署文档：https://egonlin.com/?p=10762

升级k8s

一、注意事项

（1）升级前最好备份所有组件及数据，例如etcd

（2）不要跨两个大版本进行升级，可能会存在版本bug，如：

1.19.4-->1.20.4 通常ok

1.19.4-->1.21.4 慎重

跨多个版本的可以逐个版本依次进行升级，因为下一个版本通常会考虑到针对上一个版本的升级兼容性

例如

```
1.19.4-->1.21.4

可以先从1.19.4 升级到1.20.4
然后再从1.20.4 升级到 1.21.4
```

（3）升级前要在测试环境进行充分的演练，充分考虑到回滚方案（事先将回滚动作制作成脚本，方便生产环境遇到问题时快速响应）

二、升级方案

对于线上环境，升级k8s需要做到不中断当前业务容器的灰度升级，常见方案有如下两种

- （1）方案一：蓝绿方式

仿照现有k8s集群，部署一套新环境，在新环境里部署指定版本的k8s，然后把业务应用也部署到新环境。然后将流量切换到新环境

- （2）方案二：先更新master上的k8s服务版本，再逐个批量更新Node上的k8s服务版本（高版本的master通常可以管理更低版本的node，但版本差异过大也会有问题）

蓝绿的方式没啥没说的（就是部署一套全新环境，然后切流量即可），我们直接看方案二吧

三、升级步骤

针对上述方案二，针对kubeadm方式升级k8s集群

egonlin.com

3.1、配置新的yum源

配置阿里云yum源，每台节点都需要配置（参考：https://developer.aliyun.com/mirror/kubernetes/）

下面是老版yum源配置，最多支持到k8s1.28.2-0版本

```
[root@k8s-master1 yum.repos.d]# pwd
/etc/yum.repos.d
[root@k8s-master1 yum.repos.d]# cat kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=0
repo_gpgcheck=0
gpgkey=https://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg
https://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg
```

```
kubeadm.x86_64                                  1.24.14-0                                       kubernetes
kubeadm.x86_64                                  1.24.15-0                                       kubernetes
kubeadm.x86_64                                  1.24.16-0                                       kubernetes
kubeadm.x86_64                                  1.24.17-0                                       kubernetes
kubeadm.x86_64                                  1.25.0-0                                        kubernetes
kubeadm.x86_64                                  1.25.1-0                                        kubernetes
kubeadm.x86_64                                  1.25.2-0                                        kubernetes
kubeadm.x86_64                                  1.25.3-0                                        kubernetes
kubeadm.x86_64                                  1.25.4-0                                        kubernetes
kubeadm.x86_64                                  1.25.5-0                                        kubernetes
kubeadm.x86_64                                  1.25.6-0                                        kubernetes
kubeadm.x86_64                                  1.25.7-0                                        kubernetes
kubeadm.x86_64                                  1.25.8-0                                        kubernetes
kubeadm.x86_64                                  1.25.9-0                                        kubernetes
kubeadm.x86_64                                  1.25.10-0                                       kubernetes
kubeadm.x86_64                                  1.25.11-0                                       kubernetes
kubeadm.x86_64                                  1.25.12-0                                       kubernetes
kubeadm.x86_64                                  1.25.13-0                                       kubernetes
kubeadm.x86_64                                  1.25.14-0                                       kubernetes
kubeadm.x86_64                                  1.26.0-0                                        kubernetes
kubeadm.x86_64                                  1.26.1-0                                        kubernetes
kubeadm.x86_64                                  1.26.2-0                                        kubernetes
kubeadm.x86_64                                  1.26.3-0                                        kubernetes
kubeadm.x86_64                                  1.26.4-0                                        kubernetes
kubeadm.x86_64                                  1.26.5-0                                        kubernetes
kubeadm.x86_64                                  1.26.6-0                                        kubernetes
kubeadm.x86_64                                  1.26.7-0                                        kubernetes
kubeadm.x86_64                                  1.26.8-0                                        kubernetes
kubeadm.x86_64                                  1.26.9-0                                        kubernetes
kubeadm.x86_64                                  1.27.0-0                                        kubernetes
kubeadm.x86_64                                  1.27.1-0                                        kubernetes
kubeadm.x86_64                                  1.27.2-0                                        kubernetes
kubeadm.x86_64                                  1.27.3-0                                        kubernetes
kubeadm.x86_64                                  1.27.4-0                                        kubernetes
kubeadm.x86_64                                  1.27.5-0                                        kubernetes
kubeadm.x86_64                                  1.27.6-0                                        kubernetes
kubeadm.x86_64                                  1.28.0-0                                        kubernetes
kubeadm.x86_64                                  1.28.1-0                                        kubernetes
kubeadm.x86_64                                  1.28.2-0                                        kubernetes
[root@k8s-master-01 /etc/yum.repos.d]#  yum list --showduplicates kubeadm --disableexcludes=kubernetes
```

如果需要使用 1.28 及以上版本，请使用 新版配置方法 进行配置。

新版下载地址：https://mirrors.aliyun.com/kubernetes-new/

yum源配置如下

egonlin.com

```
[root@k8s-master-01 /etc/yum.repos.d]# cat kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://mirrors.aliyun.com/kubernetes-new/core/stable/v1.30/rpm/
enabled=1
gpgcheck=1
gpgkey=https://mirrors.aliyun.com/kubernetes-
new/core/stable/v1.30/rpm/repodata/repomd.xml.key
```

配置后更新yum源，执行命令：

```
yum clean all
yum makecache
```

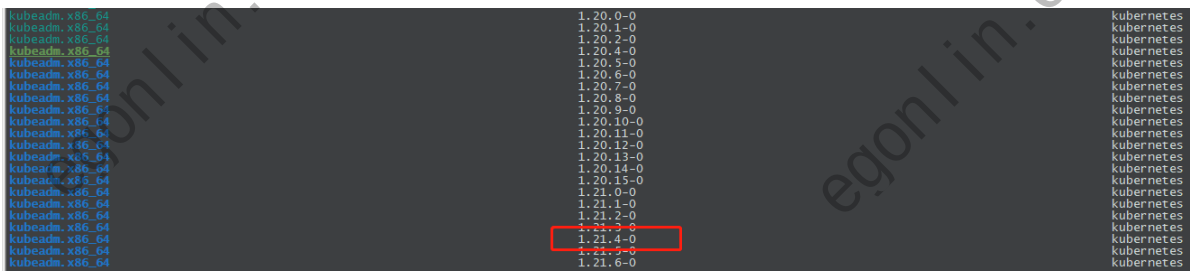
3.2、确认升级次数

查看当前版本，目前我的版本是v1.20.4

```
[root@k8s-master1 ~]# kubectl get nodes
NAME          STATUS   ROLES                  AGE   VERSION
k8s-master1   Ready    control-plane,master   41d   v1.20.4
k8s-node1     Ready    jenkins,node           41d   v1.20.4
k8s-node2     Ready    gitlab,node            38d   v1.20.4
```

基于新yum源查出目前所有正式版本

```
yum list --showduplicates kubeadm --disableexcludes=kubernetes
```



目前我的版本是v1.20.4，假设官网最新版本已经到1.25.0。升级到最新版本需要逐步升级：v1.20.4-->v1.21.4-->v1.22.4-->v1.23.4-->v1.24.4-->v1.25.0，共升级5次。

3.3、升级节点步骤综述

下面说的node包括master node与worker node

整体顺序是：先升master再升级worker

egonlin.com

升级或更新节点步骤：（其他node节点如果要升级，则采用一样的步骤）

- 1、先隔离Node节点的业务流量，备份好数据（如果是master节点则记着备份/var/lib/etcd目录）
- 2、cordon禁止新pod调度到当前node # kubectl cordon node master01
- 3、对关键服务创建PDB保护策略，确保下一步的排空时，关键服务的pod至少有1个副本可用（在当前节点以外的节点上有分布）
- 4、drain排空业务pod（静态pod不可能被排空，我们要升级的就是静态pod及kubelet等组件）
- 5、升级当前node上的软件
- 6、uncordon当前node

详细如下

下面我们来看一下。

1、迁移的带来的服务中断问题

```
当node节点关机后，k8s在等待大约6（检测（10s+40s）+300s容忍时间）分钟后，会自动将停机node节点上的pod自动迁移到其他node节点上。如果某个服务的pod只有一个副本，那么在此期间服务是中断的。


如何解决该问题？
```

2、如何避免等待默认的6分钟带来的服务中断问题

```
一方面：某一个服务的pod副本应该有多个，而且你应该在迁移前就确保这多个副本并没有完全集中在同一个节点上，如果有，那么需要提供控制调度到其他节点

另外一方面：在节点不可用之前（停机维护或者升级），将pod先驱逐到别的节点（以此规避不必要的6分钟等待）
需要用到cordon、drain、uncordor三个命令实现节点的主动维护。
```

3、cordon、drain、uncordor三个命令

```
cordon：标记节点不可调度，后续新的pod不会被调度到此节点，但是该节点上的pod可以正常对外服务；
drain：驱逐节点上的pod至其他可调度节点；
uncordon：重新标记节点可调度；


具体操作过程如下：
# 1.标记节点不可调度
# kubectl cordon k8s-3-219
node/k8s-3-219 cordoned

# 查看节点状态，此时219被标记为不可调度
# kubectl get node
NAME          STATUS                     ROLES    AGE   VERSION
k8s-3-217   Ready                      master   89d   v1.18.2
k8s-3-218   Ready                      <none>   88d   v1.18.2
k8s-3-219   Ready,SchedulingDisabled   <none>   88d   v1.18.2

# 2.驱逐pod
# kubectl drain k8s-3-219 --delete-local-data --ignore-daemonsets --force
```

egonlin.com

```
node/k8s-3-219 already cordoned
WARNING: ignoring DaemonSet-managed Pods: ingress-nginx/nginx-ingress-controller-gmzq6, kube-system/kube-flannel-ds-amd64-5gfwh, kube-system/kube-proxy-vdckk
evicting pod kube-system/tiller-deploy-6c65968d87-75pfm
evicting pod kube-system/metrics-server-7f96bbcc66-bgt7j
evicting pod test/helloworld-79956d95b4-nnlrq

# 参数如下：
--delete-local-data  删除本地数据（关键数据要提前备份好），即使emptyDir也将删除（新版本不再支持该选型）
--ignore-daemonsets  忽略DeamonSet，否则DeamonSet被删除后，仍会自动重建；
--force  不加force参数只会删除该node节点上的ReplicationController, ReplicaSet, DaemonSet,StatefulSet or Job，加上后所有pod都将删除；

# 3. 查看驱逐，219上的pod迁移到218上了。
# kubectl get pod -n test -o wide
NAME                          READY   STATUS        RESTARTS   AGE   IP
NODE
helloworld-79956d95b4-gg58c   0/1     Running       0          20s   10.244.1.165
  k8s-3-218
helloworld-79956d95b4-nnlrq   1/1     Terminating   0          77m   10.244.2.215
  k8s-3-219



如何能够做到平滑迁移呢？我们继续往下看。
```

3、添加PDB保护策略：（虽然我们建议一个关键的不可中断服务通常应该至少两副本，但实际情况可能不会这么规范，此时如果你们部门的小弟执行了上面的步骤排空了pod导致了过程中服务不可用这个锅就是你的了，我们可以添加PDB策略来对一些关键服务进行硬性限制，加了之后再去drain排空的时候，如果存在一些受PDB保护的应用，则会drain失败，此时会硬性要求你先扩容副本，因为你已经cordon过了，再启新副本一定是在新节点上）

```
要做到平滑迁移就需要用的pdb(PodDisruptionBudget)，即主动驱逐保护：实现节点维护期间不低于一定数量的pod正常运行，从而保证服务的可用性


在仍以helloworld为例，由于只有一个副本，因此需要保证维护期间这个副本在迁移完成后，才会终止。

# 从218 驱逐到 219
# 1.标记节点不可调度
# kubectl cordon k8s-3-218
node/k8s-3-218 cordoned



# 2.新建pdb
vim pdb-test.yaml
apiVersion: policy/v1beta1
kind: PodDisruptionBudget
metadata:
  name: pdb-test
  namespace: test
spec:
  minAvailable: 1
  selector:
    matchLabels:
```

```
      app: helloworld



# 3.应用并查看状态
# kubectl apply -f pdb-test.yaml
# kubectl get pdb -n test
NAME       MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
pdb-test
1                          N/A                          0
              7s




# 4.驱逐
# kubectl drain k8s-3-218 --delete-local-data --ignore-daemonsets --force
node/k8s-3-218 already cordoned
WARNING: ignoring DaemonSet-managed Pods: ingress-nginx/nginx-ingress-controller-
hhb6h, kube-system/kube-flannel-ds-amd64-pb4d7, kube-system/kube-proxy-rzdcj
evicting pod kube-system/tiller-deploy-6c65968d87-ktqmm
evicting pod kube-system/metrics-server-7f96bbcc66-6p6wm
evicting pod test/helloworld-79956d95b4-gg58c
error when evicting pod "helloworld-79956d95b4-gg58c" (will retry after 5s):
Cannot evict pod as it would violate the pod's disruption budget.
evicting pod test/helloworld-79956d95b4-gg58c
error when evicting pod "helloworld-79956d95b4-gg58c" (will retry after 5s):
Cannot evict pod as it would violate the pod's disruption budget.
pod/tiller-deploy-6c65968d87-ktqmm evicted

#此时由于只有一个副本，最小可用为1，则ALLOWED就为0，因此在一个副本通过pdb是不会发生驱逐的。我
们需要先扩容，将副本数调整为大于1。




# 5.手动调整副本数，将replicas为2
kubectl edit deploy helloworld -n test
# 再次查看pdb
# kubectl get pdb -n test
NAME       MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
pdb-test   1               N/A               1                     13m

#此时最小可用为1 ，ALLOWED为1，此时是数量会自动计算。




# 6.驱逐
# kubectl drain k8s-3-218 --delete-local-data --ignore-daemonsets --force

# 7.维护完毕，将node调整为可调度
# kubectl uncordon k8s-3-218

本次驱逐，由于helloworld始终保持有一个pod在提供服务，因此服务是不中断的。
最后将副本数可以调节为1并将node节点调整为可调度，维护完毕。
```

3.4、开始升级---升级 master 节点

- 1、先隔离Node节点的业务流量，备份好数据（如果是master节点则记着备份/var/lib/etcd目录）
- 2、cordon禁止新pod调度到当前node

```
kubectl cordon k8s-master1
```

- 3、对关键服务创建PDB保护策略(无则略)
- 4、drain排空业务pod（静态pod不可能被排空，我们要升级的就是静态pod及kubelet等组件）

```
 kubectl drain k8s-master1 --delete-local-data --ignore-daemonsets --force
```

- 5、升级当前node上的软件

  详细步骤见下面
- 6、如果master节点是高可用集群，那么需要在当前master节点之外的其他master节点上执行下述命令

```
kubeadm upgrade node
```

- 7、uncordon当前node

```
kubectl uncordon k8s-master1
```

（1）在 master 节点上执行如下命令，升级 kubeadm，对应的版本必须一致v1.21.4

如果是高可用集群，找其中一台master执行以下命令

```
[root@k8s-master1 ~]# yum install -y kubeadm-1.21.4-0 --
disableexcludes=kubernetes


Upgrade  1 Package

Total download size: 9.1 M
Downloading packages:
Delta RPMs disabled because /usr/bin/applydeltarpm not installed.
61c56c520cec529ff02ca33f37f190d23253acff6e84bd695cc045cdd4f52b2e-kubeadm-1.21.4-
0.x86_64.rpm                                                | 9.1 MB  00:01:24

Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
```

egonlin.com

```
  Updating   : kubeadm-1.21.4-0.x86_64

 1/2
  Cleanup    : kubeadm-1.20.4-0.x86_64

 2/2
  Verifying  : kubeadm-1.21.4-0.x86_64

 1/2
  Verifying  : kubeadm-1.20.4-0.x86_64

 2/2

Updated:
  kubeadm.x86_64 0:1.21.4-0



Complete!
```

（2）在同一台master上继续继续以下命令，kubeadm更新计划

```
[root@k8s-master1 ~]# kubeadm upgrade plan
```

kubeadm更新计划会打印出目前能支持到的版本，显示我最高可以升级到v1.21.14，由于我们当前安装的kubeadm工具是v1.21.4版本，后续也必须保持统一版本，如果这里apply了1.21.14会报错

在打印信息中可以看到，升级集群每个组件对应的当前版本和升级后的版本。而且升级的组件只包括**kube-apiserver，kube-controller-manager，kube-scheduler，kube-proxy，CoreDNS，etcd。**

不包括kubectl,kubelet,docker和网络组件flannel等

```
_____________________________________________________________________
Components that must be upgraded manually after you have upgraded the control plane with 'kubeadm upgrade apply':
COMPONENT   CURRENT       TARGET
kubelet     3 x v1.20.4   v1.21.14

Upgrade to the latest stable version:

COMPONENT                 CURRENT    TARGET
kube-apiserver            v1.20.4    v1.21.14
kube-controller-manager   v1.20.4    v1.21.14
kube-scheduler            v1.20.4    v1.21.14
kube-proxy                v1.20.4    v1.21.14
CoreDNS                   1.7.0      v1.8.0
etcd                      3.4.13-0   3.4.13-0

You can now apply the upgrade by executing the following command:

        kubeadm upgrade apply v1.21.14

Note: Before you can perform this upgrade, you have to update kubeadm to v1.21.14.

_____________________________________________________________________
```

（3）根据上面的输出提示，执行如下命令以升级

```
[root@k8s-master1 ~]# kubeadm upgrade apply v1.21.4
....
[bootstrap-token] configured RBAC rules to allow Node Bootstrap tokens to get nodes
```

egonlin.com

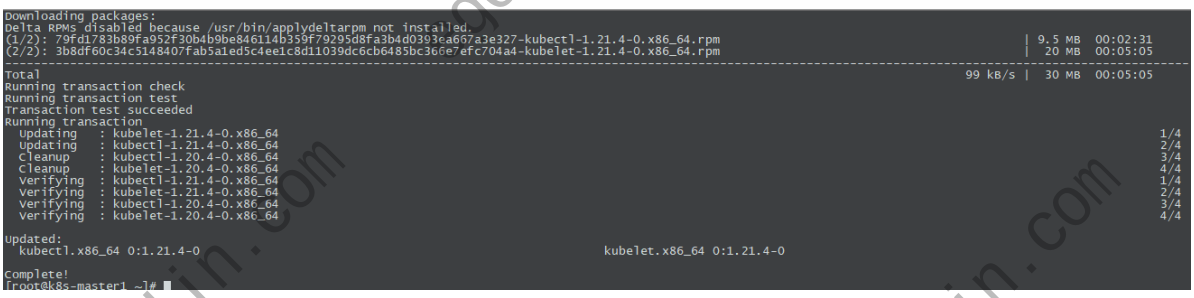
```
[bootstrap-token] configured RBAC rules to allow Node Bootstrap tokens to post
CSRs in order for nodes to get long term certificate credentials
[bootstrap-token] configured RBAC rules to allow the csrapprover controller
automatically approve CSRs from a Node Bootstrap Token
[bootstrap-token] configured RBAC rules to allow certificate rotation for all
node client certificates in the cluster
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.21.4". Enjoy!

[upgrade/kubelet] Now that your control plane is upgraded, please proceed with
upgrading your kubelets if you haven't already done so.
#升级成功
```

如果是高可用还需要在其他 master 节点上执行命令：（否则会导致其上的pod无法正确启动，如果你之前没有做过cordon排空那该节点上的pod就会处于CreateContainerConfigError状态，执行下述命令才可以恢复正常）

```
kubeadm upgrade node
```

在所有的 master 节点上执行如下命令升级 kubelet 和 kubectl

```
[root@k8s-master1 ~]# yum install -y kubelet-1.21.4-0 kubectl-1.21.4-0 --
disableexcludes=kubernetes
```



执行如下命令，以重启 kubelet（重启后才会看到加载了新版本）

```
[root@k8s-master1 ~]# systemctl daemon-reload && systemctl restart kubelet
```

3.4、升级node节点

前提：升级节点的软件，或者是硬件例如增大内存，都会涉及到的节点的暂时不可用，这与停机维护是一个问题，我们需要考虑的核心问题是节点不可用过程中，如何确保节点上的pod服务不中断！！！

升级或更新node节点步骤：（其他node节点如果要升级，则采用一样的步骤）

- 1、先隔离Node节点的业务流量
- 2、cordon禁止新pod调度到当前node

egonlin.com

```
kubectl cordon node2
```

- 3、对关键服务创建PDB保护策略，确保下一步的排空时，关键服务的pod至少有1个副本可用（在当前节点以外的节点上有分布）
- 4、drain排空pod

```
kubectl drain node2 --delete-local-data --ignore-daemonsets --force
```

- 5、升级当前node上的软件

5.1 在所有的 node 节点上执行如下命令，升级 kubeadm

```
[root@k8s-node2 ~]# yum install -y kubeadm-1.21.4-0 --
disableexcludes=kubernetes
```

```
 kubeadm                                 x86_64                                 1.21.4-0                               kubernetes                              9.1 M

Transaction Summary
================================================================================================================================================================
Upgrade  1 Package

Total download size: 9.1 M
Downloading packages:
Delta RPMs disabled because /usr/bin/applydeltarpm not installed.
61c56c520cec529ff02ca33f37f190d23253acff6e84bd695cc045cdd4f52b2e-kubeadm-1.21.4-0.x86_64.rpm                                          | 9.1 MB  00:01:36
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Updating   : kubeadm-1.21.4-0.x86_64                                                                                                                        1/2
  Cleanup    : kubeadm-1.20.4-0.x86_64                                                                                                                        2/2
  Verifying  : kubeadm-1.21.4-0.x86_64                                                                                                                        1/2
  Verifying  : kubeadm-1.20.4-0.x86_64                                                                                                                        2/2

Updated:
  kubeadm.x86_64 0:1.21.4-0
```

5.2 升级 kubelet 的配置，在所有node节点上执行

```
[root@k8s-node2 ~]# kubeadm upgrade node
[upgrade] Reading configuration from the cluster...
[upgrade] FYI: You can look at this config file with 'kubectl -n kube-system
get cm kubeadm-config -o yaml'
[preflight] Running pre-flight checks
[preflight] Skipping prepull. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[kubelet-start] Writing kubelet configuration to file
"/var/lib/kubelet/config.yaml"
[upgrade] The configuration for this node was successfully updated!
[upgrade] Now you should go ahead and upgrade the kubelet package using your
package manager.
```

5.3 升级 kubelet 和 kubectl

```
[root@k8s-node2 ~]# yum install -y kubelet-1.21.4-0 kubectl-1.21.4-0 --
disableexcludes=kubernetes
```

```
Downloading packages:
Delta RPMs disabled because /usr/bin/applydeltarpm not installed.
(1/2): 79fd1783b89fa952f30b4b9be846114b350f79395d8fa3b4d0393ea667a3e327-kubectl-1.21.4-0.x86_64.rpm                                    | 9.5 MB  00:01:40
(2/2): 3b8df60c34c514840f7ab5a1ed5c4ee1c8d110396dc6cb6485bc366e7efc704a4-kubelet-1.21.4-0.x86_64.rpm                                    |  20 MB  00:03:29
--------------------------------------------------------------------------------------------------------------------------------------------------------------
Total                                                                                                                   145 kB/s |  30 MB  00:03:29
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Updating   : kubelet-1.21.4-0.x86_64                                                                                                                        1/4
  Updating   : kubectl-1.21.4-0.x86_64                                                                                                                        2/4
  Cleanup    : kubectl-1.20.4-0.x86_64                                                                                                                        3/4
  Cleanup    : kubelet-1.20.4-0.x86_64                                                                                                                        4/4
  Verifying  : kubectl-1.21.4-0.x86_64                                                                                                                        1/4
  Verifying  : kubelet-1.21.4-0.x86_64                                                                                                                        2/4
  Verifying  : kubectl-1.20.4-0.x86_64                                                                                                                        3/4
  Verifying  : kubelet-1.20.4-0.x86_64                                                                                                                        4/4

Updated:
  kubectl.x86_64 0:1.21.4-0                                                   kubelet.x86_64 0:1.21.4-0

Complete!
[root@k8s-node2 ~]#
```

执行如下命令，以重启 kubelet

```
[root@k8s-node2 ~]# systemctl daemon-reload && systemctl restart kubelet
```

- 6、uncordon当前node

```
kubectl uncordon node2
```

3.5、检查集群的状态

版本已经从v1.20.4升级到v1.21.4

```
[root@k8s-master1 ~]# kubectl get nodes
NAME          STATUS   ROLES                  AGE   VERSION
k8s-master1   Ready    control-plane,master   41d   v1.21.4
k8s-node1     Ready    jenkins,node           41d   v1.21.4
k8s-node2     Ready    gitlab,node            38d   v1.21.4
```

如果是二进制安装的k8s，那升级无非就是下载新版本的软件包，然后启系统服务，步骤是类似的

升级注意事项：https://egonlin.com/?p=10757

egonlin.com